

Testing (Practice)

Week 8

Quality is not an act, it is a habit.

— Aristotle

Objectives

- Obtain test coverage in Android Studio
 - Learn how to write integration tests for Fragments
 - Espresso basics
 - Using mockito with espresso
-
- | | | |
|--------|---|---------------------------------------|
| Done | { | ● Unit tests |
| | | ● Integrated tests |
| Today | { | ○ Screen UI tests Fragment |
| | | ○ User flow tests or navigation tests |
| Future | { | ● Usability tests |
| | | ○ Heuristic Evaluation (HE) |
| | | ○ User Acceptance Test (UAT) |

Skeleton Code

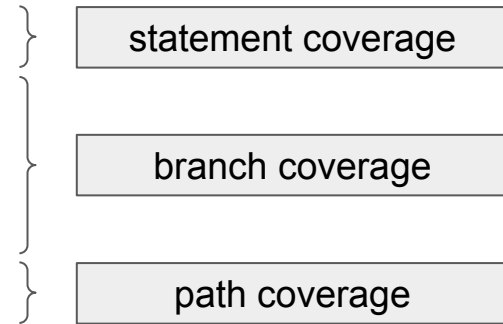
Git Link: <https://github.com/snuhcs-course/swpp-practice-testing-todo-app/>

- testing2/start
 - no test code (start with this!)
- testing2/end
 - test code dealt in class included (for reference)

Test Coverage and Code Quality Metrics

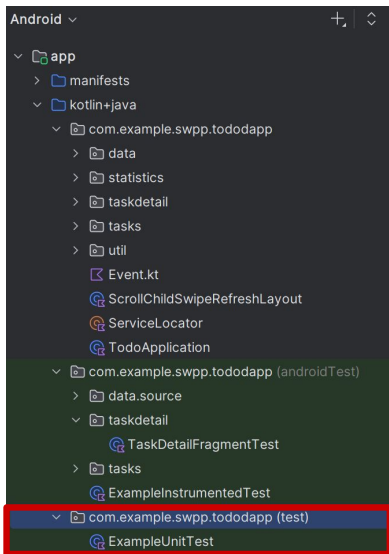
Recap: Test Coverage

- Coverage = % of covered code of total available code
- Considered “covered” if the target code is executed at least once by at least one test
- Types based on coverage unit
 - C0: every statement (line of code)
 - C1: every branch
 - C1+ decision coverage: every subexpression in conditional
 - C2: every path

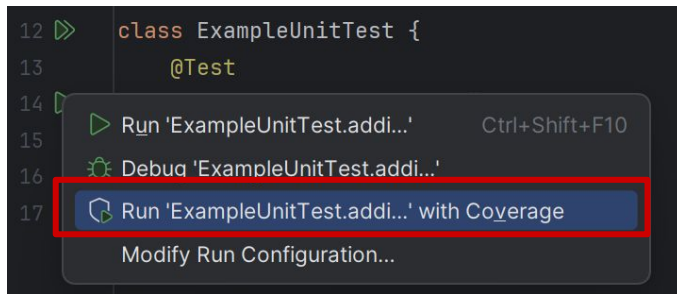


Test Coverage Report in Android Studio

- We can get statement coverage for unit tests in Android Studio
- Open ExampleUnitTest.kt under the com.example.swpp.tododapp (test) package
- Select “Run ‘test_name’ with Coverage”



```
12 >> class ExampleUnitTest {  
13     @Test  
14     fun addition_isCorrect() {  
15         assertEquals( expected = 4, 2 + 2)  
16     }  
17 }
```



Coverage Report in Android Studio

Coverage ExampleUnitTest.addition_isCorrect ×				
statement coverage				
Element ^	Class, %	Method, %	Line, %	Branch, %
all	0% (0/125)	0% (0/386)	0% (0/1605)	0% (1/615)
androidx.databinding	0% (0/2)	0% (0/1)	0% (0/2)	100% (0/0)
library.baseAdapters	0% (0/1)	100% (0/0)	100% (0/0)	100% (0/0)
DataBinderMapperImpl	0% (0/1)	0% (0/1)	0% (0/2)	100% (0/0)
DataBindingComponent	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
com.example.swpp.tododapp	0% (0/123)	0% (0/385)	0% (0/1603)	0% (1/615)
addedittask	0% (0/11)	0% (0/22)	0% (0/75)	0% (0/38)
data	0% (0/47)	0% (0/141)	0% (0/347)	1% (1/52)
databinding	0% (0/10)	0% (0/79)	0% (0/725)	0% (0/345)
generated.callback	0% (0/1)	0% (0/2)	0% (0/4)	100% (0/0)
statistics	0% (0/10)	0% (0/15)	0% (0/41)	0% (0/26)
taskdetail	0% (0/12)	0% (0/32)	0% (0/89)	0% (0/40)
tasks	0% (0/20)	0% (0/60)	0% (0/206)	0% (0/65)
util	0% (0/1)	0% (0/4)	0% (0/15)	0% (0/4)
BR	0% (0/1)	100% (0/0)	100% (0/0)	100% (0/0)
BuildConfig	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)
DataBinderMapperImpl	0% (0/3)	0% (0/8)	0% (0/56)	0% (0/31)
Event	0% (0/1)	0% (0/4)	0% (0/7)	0% (0/2)
EventObserver	0% (0/1)	0% (0/2)	0% (0/4)	0% (0/2)
ScrollChildSwipeRefreshLayout	0% (0/1)	0% (0/3)	0% (0/4)	0% (0/2)
ServiceLocator	0% (0/2)	0% (0/9)	0% (0/25)	0% (0/6)
TodoApplication	0% (0/1)	0% (0/3)	0% (0/4)	0% (0/2)

Statement Coverage

- You can see which statement is covered and which is not
 - Green: executed during test
 - Red: not executed

Example from another project



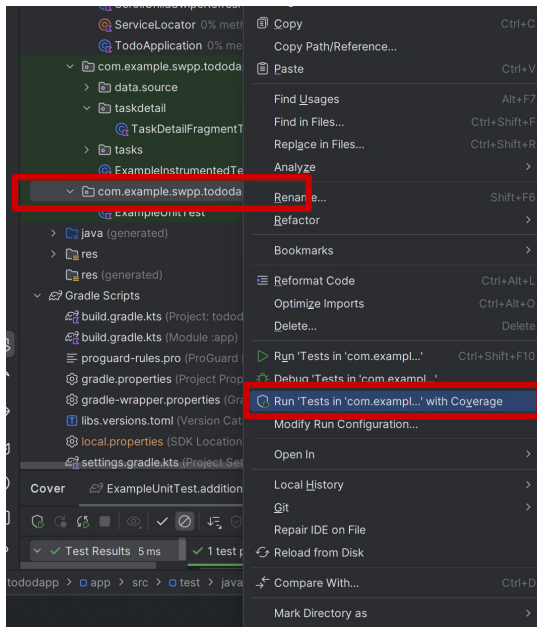
The screenshot shows a code editor with Java code. On the left side of the code, there is a vertical line with markers indicating statement coverage. Green markers indicate statements that were executed during a test, while red markers indicate statements that were not executed. The code is as follows:

```
10 public RollDiceUseCase(NumRollRepository numRollRepository, RollDiceUtils rollDiceUtils) {  
11     this.numRollRepository = numRollRepository;  
12     this.rollDiceUtils = rollDiceUtils;  
13 }  
14  
15 public void rollDice() {  
16     firstDieValue = rollDiceUtils.getRandomDiceNumber();  
17     secondDieValue = rollDiceUtils.getRandomDiceNumber();  
18     primeNumbersSmallerThanProduct = rollDiceUtils.findPrimeNumbers(firstDieValue, secondDieValue);  
19     notifyRoll();  
20 }  
21 private void notifyRoll() { numRollRepository.incNumRoll(); }  
24 public void notifyReset() { numRollRepository.resetNumRoll(); }
```

The coverage markers on the left are: a green bar for line 10, a green bar for line 11, a green bar for line 12, a green bar for line 13, a green bar for line 15, a green bar for line 16, a green bar for line 17, a green bar for line 18, a green bar for line 19, a green bar for line 20, a green bar for line 21, and a red bar for line 24.

Run All Unit Tests

- To run all unit tests at once
 - Right-click `com.example.package_name` (test) folder
 - Run tests in '`com.example.package_name`' with Coverage

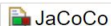


Homework 1: Test Coverage for your Project

- Run all the unit tests for your project
- Get test coverage report
- Submit a screenshot of your report
 - Actual coverage doesn't matter
- Individual submission for each team member

Jacoco

- Free test coverage library for JVM-based projects
 - Calculates types of coverage metrics (line coverage, branch coverage, method coverage, and class coverage)
 - Generates detailed HTML reports to visualize test coverage



JaCoCo

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 org.jacoco.core		97%		92%	121	1,447	122	3,459	20	723	2	142
 org.jacoco.examples		58%		64%	24	53	97	193	19	38	6	12
 org.jacoco.agent.rt		75%		83%	32	130	75	344	21	80	7	22
 jacoco-maven-plugin		90%		82%	35	194	49	466	8	117	0	23
 org.jacoco.cli		97%		100%	4	109	10	275	4	74	0	20
 org.jacoco.report		99%		99%	4	572	2	1,345	1	371	0	64
 org.jacoco.ant		98%		99%	4	163	8	429	3	111	0	19
 org.jacoco.agent		86%		75%	2	10	3	27	0	6	0	1
Total	1,431 of 28,088	94%	160 of 2,252	92%	226	2,678	366	6,538	76	1,520	15	303

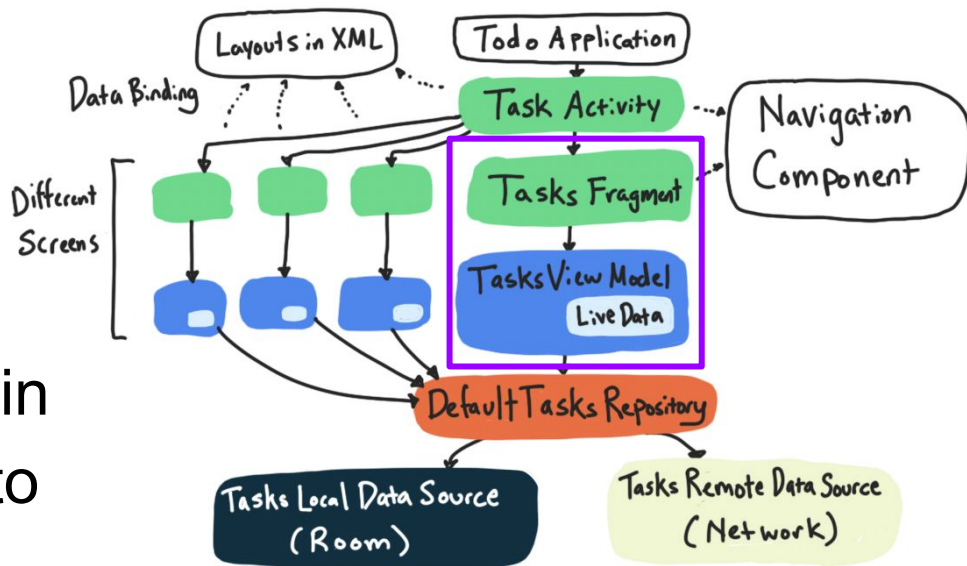
Integration Testing with Espresso

Integration Tests: Recap

- Test the interaction of several classes to make sure they behave as expected when used together
- Run as instrumentation tests (androidTest source set)

Today's Goal

- Verify that an active task's details appear when its fragment is opened.
- Verify that selecting a task in TasksFragment navigates to its detail screen.



Our First Integration Test!

- GIVEN: An active task exists in the repository
- WHEN: The TaskDetailFragment is launched
- THEN: The UI correctly displays:
 - Title
 - Description
 - Completion status

Needs to look at UI elements
⇒ use **Espresso**

Espresso

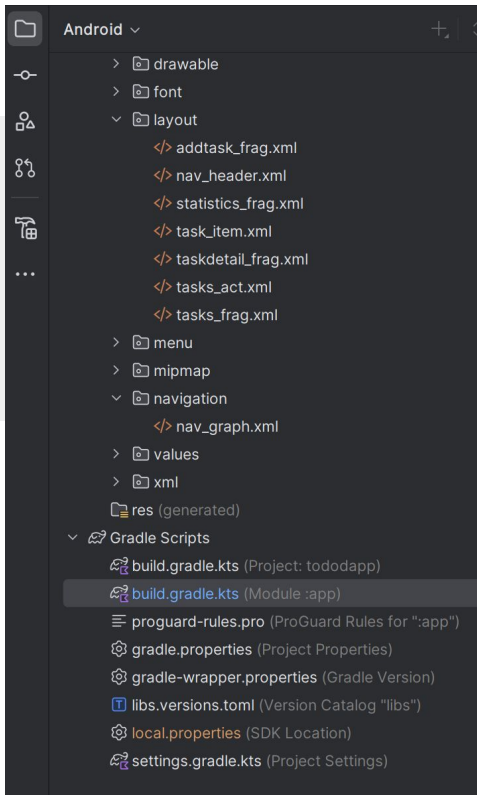
- Google framework for UI testing (mainly white-box testing)
- Simulate user interactions with Views
- Key features
 - **Synchronization**: waits until main thread is idle before running UI test actions to ensure stable state when performing commands
 - **Clean syntax**: simple and readable test code
 - Supports wide range of **user interactions** (click button, swipe, etc.)
 - **Compatible with Android** development tools and build system
- For those using *jetpack compose*, please refer to the official doc:
<https://developer.android.com/develop/ui/compose/testing>

Espresso Setup: Add Dependency

- Add dependency to **build.gradle.kts (:app)**

Already done!

```
androidTestImplementation(libs.androidx.test.espresso.core)
androidTestImplementation(libs.androidx.test.espresso.contrib)
androidTestImplementation(libs.androidx.test.espresso.intents)
androidTestImplementation(libs.androidx.test.espresso.idling.resources)
androidTestImplementation(libs.androidx.test.espresso.idling.concurrent)
```

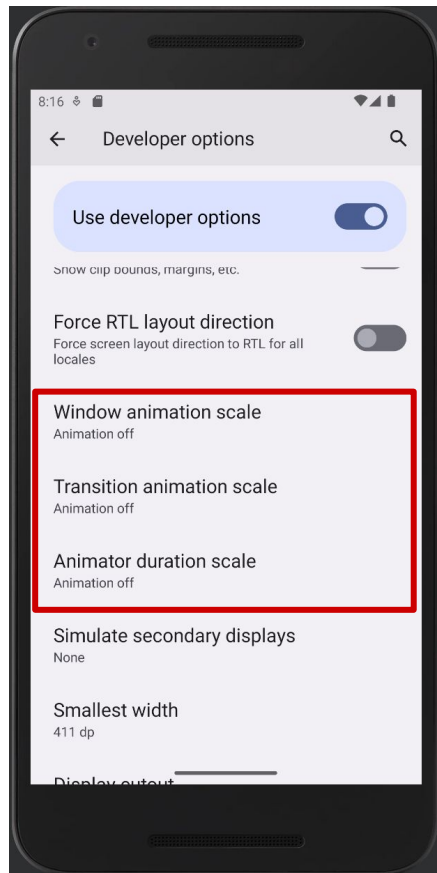


Espresso Setup: Animations Off (1/3)

- Espresso tests are instrumented tests that run on a device
- Animations make tests flaky (nondeterministic) and slow
 - Introduces delays and unpredictable timing into your app's UI
 - Espresso may attempt to interact with UI elements that are not yet visible or in a transition state, leading to test failures

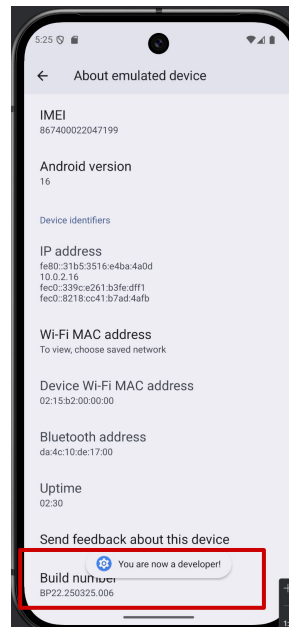
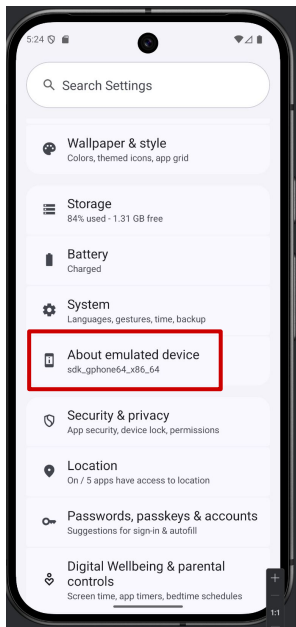
Espresso Setup: Animations Off (2/3)

- Settings → System → Developer options
- Disable
 - Window animation scale
 - Transition animation scale
 - Animator duration scale



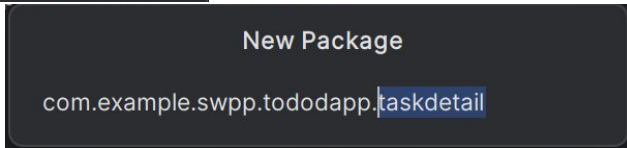
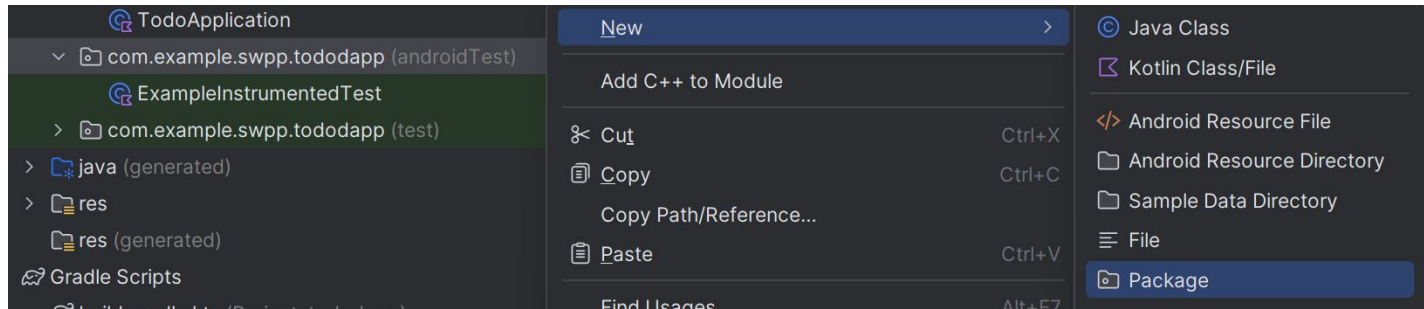
Espresso Setup: Animations Off (3/3)

- If you can't see Developer options
 - Click on "About emulated device"
 - Click on "Build number" until "You are now a developer!"



Let's Create Our First Integration Test! (1/3)

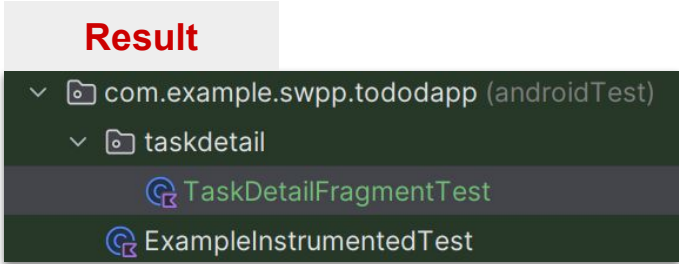
1. Right click on `com.example.swpp.tododapp` (androidTest)
2. New
3. Package
4. Append “`taskdetail`” at the end
5. Enter



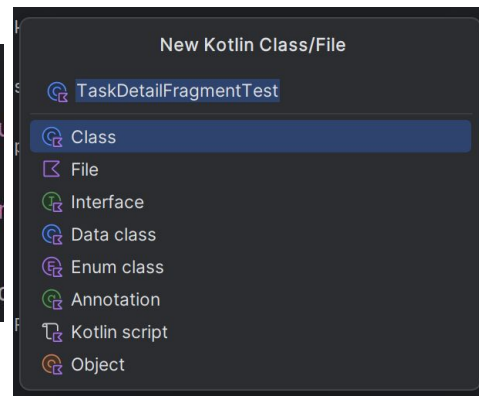
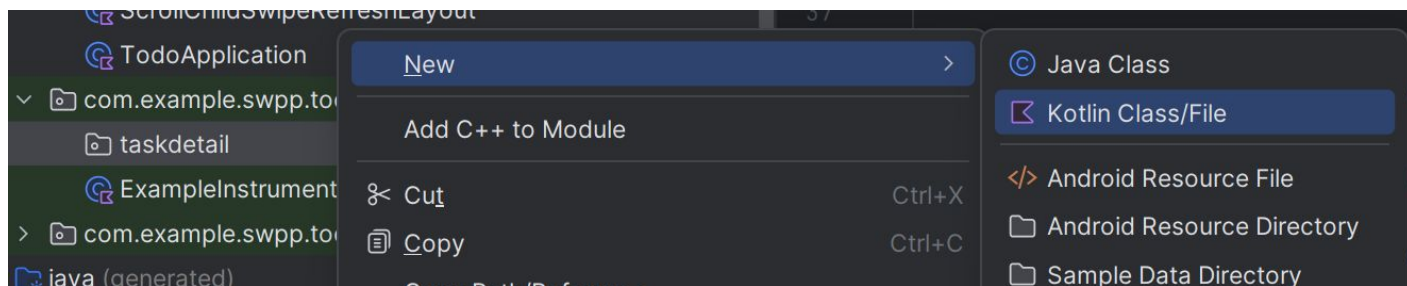
Let's Create Our First Integration Test! (2/3)

1. Right click the `taskdetail` package
2. New
3. Kotlin Class/File
4. Write `TaskDetailFragmentTest`
5. Enter

Result



```
com.example.swpp.tododapp (androidTest)
└── taskdetail
    ├── TaskDetailFragmentTest
    └── ExampleInstrumentedTest
```



Let's Create Our First Integration Test! (3/3)

Tags this test as integrated test (useful for CI/CD)

TaskDetailFragmentTest.kt



```
@MediumTest
@RunWith(AndroidJUnit4::class)
class TaskDetailFragmentTest {

    @Test
    fun activeTaskDetails_DisplayedInUi() = runTest{
        // GIVEN - Add active (incomplete) task to the DB

        // WHEN - Details fragment launched to display task

        // THEN - Task details are displayed on the screen
        // make sure that the title/description are both shown and correct
        // and make sure the "active" checkbox is shown unchecked
    }
}
```

Setup FakeAndroidTestRepository

```
class TaskDetailFragmentTest {  
    private lateinit var repository: TasksRepository  
  
    @Before  
    fun initRepository() {  
        repository = FakeAndroidTestRepository()  
        ServiceLocator.tasksRepository = repository  
    }  
  
    @After  
    fun cleanupDb() = runTest {  
        ServiceLocator.resetRepository()  
    }  
    // ...  
}
```

Setup method:

- Stores a FakeAndroidTestRepository in our ServiceLocator (a central registry for storing/retrieving shared dependencies – an alternative for dependency injection)

Tear down method:

Reset the ServiceLocatory's stored repository

GIVEN – Add an Active Task to the DB

1. Create a Task
2. Save it in our repository
3. saveTask is a coroutine suspend function, so it needs a coroutine test scope
⇒ wrap our test with runTest

```
fun activeTaskDetails_DisplayedInUi() = runTest {  
    // GIVEN - Add active task to the DB  
    val activeTask = Task("Active Task", "AndroidX Rocks", false)  
    repository.saveTask(activeTask)  
    // ...  
}
```

WHEN – Launch TaskDetailFragment (1/2)

To launch a fragment, we use `launchFragmentInContainer`

Our fragment uses `navArgs`, which has one argument of type string named `taskId`

TaskDetailFragment.kt

```
class TaskDetailFragment : Fragment() {  
    3 Usages  
    private lateinit var viewDataBinding: TaskdetailFragBinding  
  
    2 Usages  
    private val args: TaskDetailFragmentArgs by navArgs()
```

res/navigation/nav_graph.xml

```
<fragment  
    android:id="@+id/task_detail_fragment_dest"  
    android:name="com.example.swpp.tododapp.taskdetail.TaskDetailFragment"  
    android:label="Task Details">  
    <action  
        android:id="@+id/action_taskDetailFragment_to_addEditTaskFragment"  
        app:destination="@id/add_edit_task_fragment_dest" />  
    <argument  
        android:name="taskId"  
        app:argType="string" />  
    <action  
        android:id="@+id/action_taskDetailFragment_to_tasksFragment"  
        app:destination="@id/tasks_fragment_dest" />  
</fragment>
```

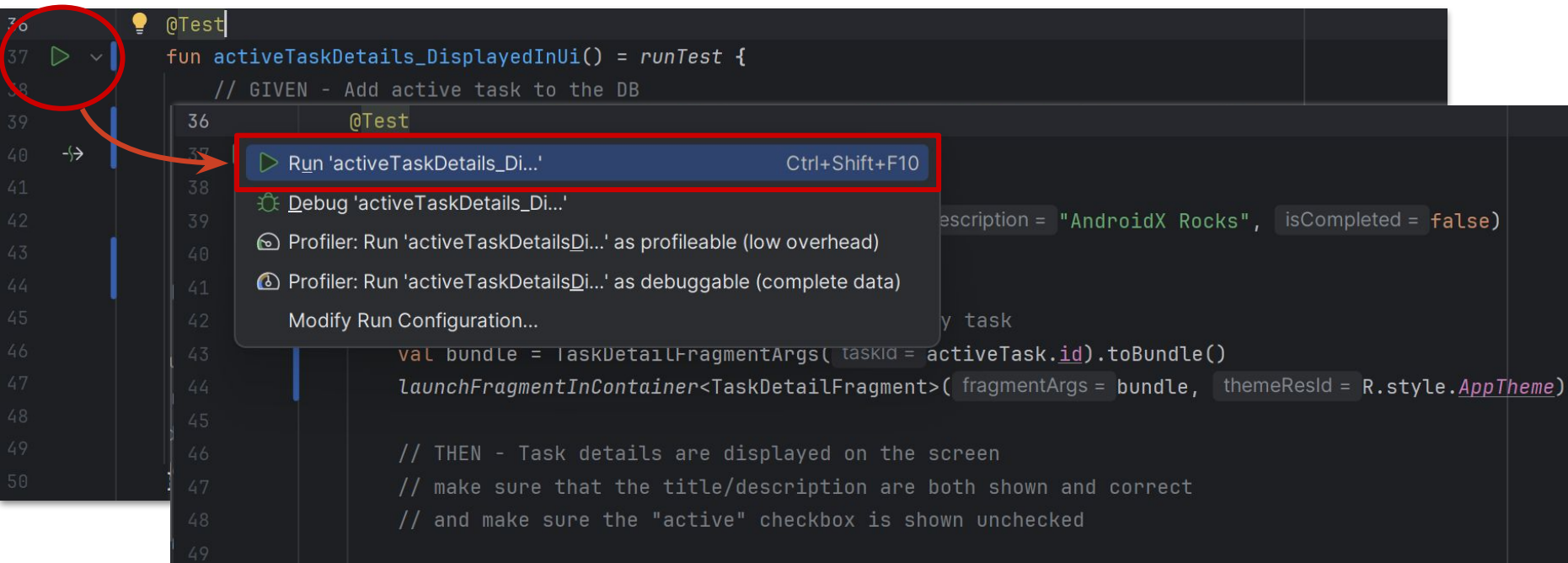
WHEN – Launch TaskDetailFragment (2/2)

Create a TaskDetailFragmentArgs with our task's id, convert it to a Bundle so that it can be passed to the launchFragmentInContainer function

```
// WHEN - Details fragment launched to display task  
val bundle = TaskDetailFragmentArgs(activeTask.id).toBundle()  
launchFragmentInContainer<TaskDetailFragment>(bundle, R.style.AppTheme)
```

Let's run this!

Click on > button and press Run



Result

- Instrumented test $\Rightarrow$ emulator / physical device is turned on
- Doesn't navigate through any other fragments, but just opens fragment and then (immediately) shuts down.
- To make the launched fragment stay longer, add sleep function.

The screenshot shows the Android Studio interface with the 'Run' tab selected. The test results for 'activeTaskDetails_DisplayedInUi()' are displayed. The test passed successfully on a Pixel_9 emulator. The build was successful in 1m 13s, and 75 actionable tasks were executed, with 58 up-to-date.

Tests	Duration	Pixel_9
✓ Test Results	1 s	1/1
✓ TaskDetailFragmentTest	1 s	1/1
✓ activeTaskDetails_DisplayedInUi	1 s	✓

Build Analyzer results available

Gradle build finished in 1 m 14 s 107 ms



Let's move to checking the UI with Espresso

- Goal: Check that when “an active task’s fragment is launched, its details are displayed on screen”.
- GIVEN: An active task exists in the repository ✓
- WHEN: The TaskDetailFragment is launched ✓
- THEN: The UI correctly displays:
 - Title
 - Description
 - Completion status



Espresso

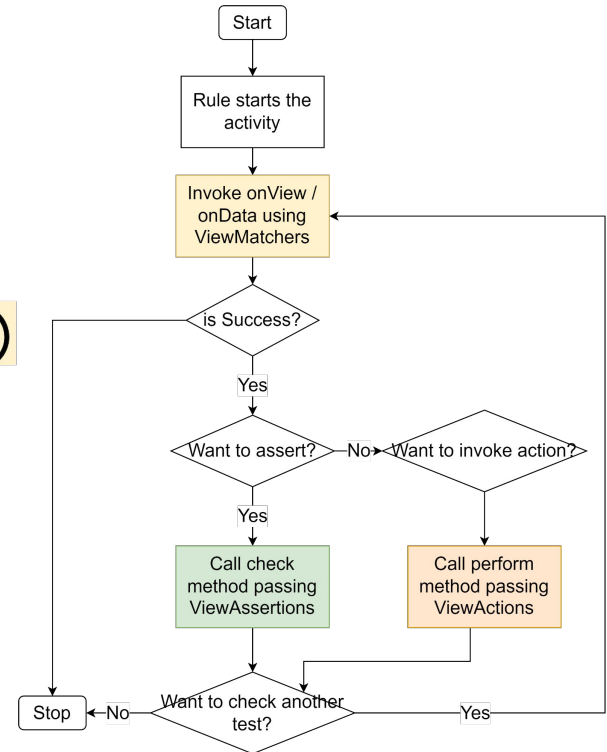
Espresso Test Example

- Find the checkbox with the id `complete_checkbox`, click it and assert that it is checked

Static Espresso function ViewMatcher

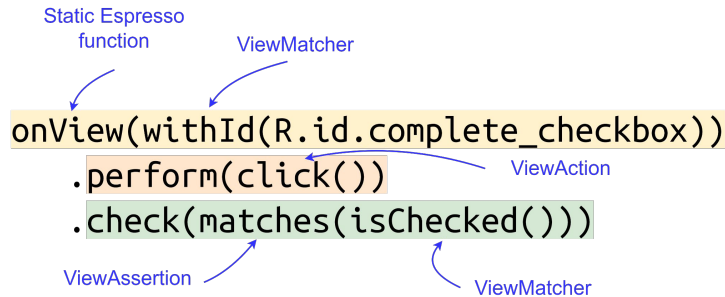
```
onView(withId(R.id.complete_checkbox))  
    .perform(click())  
    .check(matches(isChecked()))
```

ViewAction ViewAssertion ViewMatcher

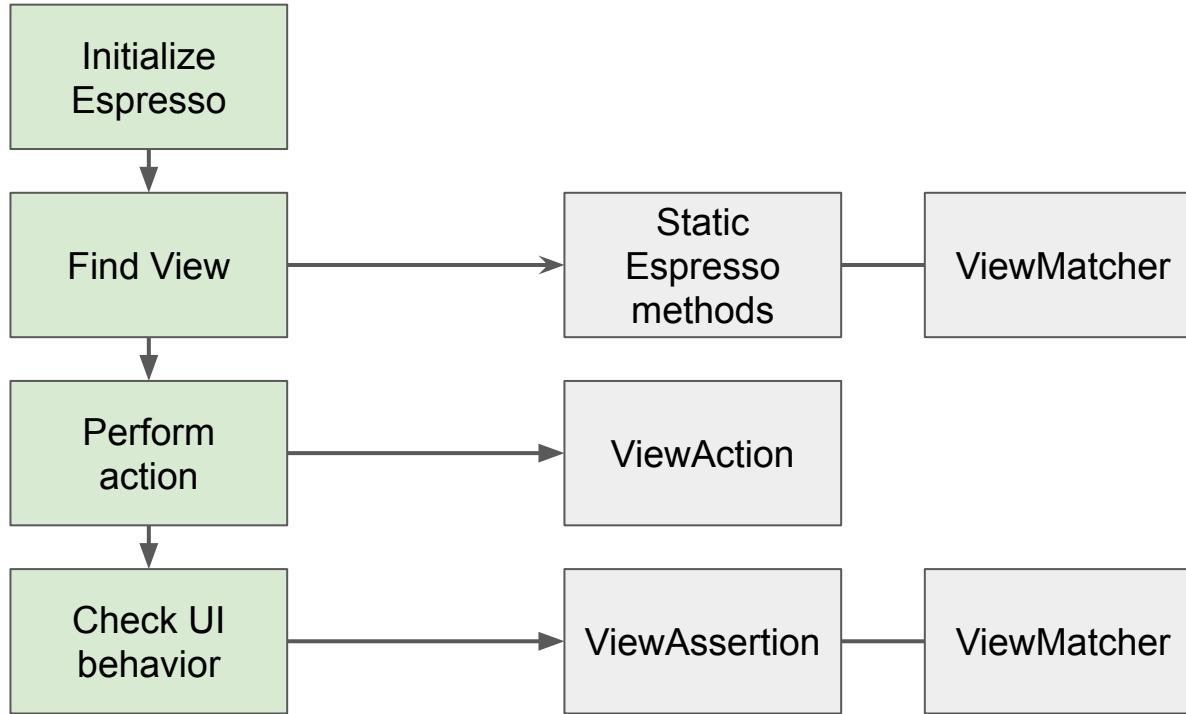


Espresso Test Components

- Static Espresso method
 - Entry point to Espresso framework
- ViewMatcher
 - Identify and select specific UI elements within the app's UI based on their properties
- ViewAction
 - Represents an action that you want to perform on a UI element
- ViewAssertion
 - Verify state or properties of UI element after performing an action



Espresso Test Workflow



Write espresso UI test code (1/4)

Goal 1: Make sure that the **title** is shown and correct

```
onView(withId(R.id.task_detail_title_text)).check(matches(isDisplayed()))
```

```
onView(withId(R.id.task_detail_title_text)).check(matches(withText("Task Title")))
```

Write espresso UI test code – exercise (2/4)

Goal 2: Make sure that the **description** is shown and correct

Answer on next page

Write espresso UI test code – answer (3/4)

Goal 2: Make sure that the **description** is shown and correct

```
onView(withId(R.id.task_detail_description_text)).check(matches(isDisplayed()))
```

```
onView(withId(R.id.task_detail_description_text)).check(matches(withText("Task  
Description")))
```

Write espresso UI test code (4/4)

Goal 3: Make sure the **checkbox** is shown & unchecked

```
onView(withId(R.id.task_detail_complete_checkbox)).check(matches(isDisplayed()))  
onView(withId(R.id.task_detail_complete_checkbox)).check(matches(not(isChecked())))
```

Full test code

```
@Test
fun activeTaskDetails_DisplayedInUi() = runTest {
    // GIVEN - Add active task to the DB
    val activeTask = Task("Task Title", "Task Description", false)
    repository.saveTask(activeTask)

    // WHEN - Details fragment launched to display task
    val bundle = TaskDetailFragmentArgs(activeTask.id).toBundle()
    launchFragmentInContainer<TaskDetailFragment>(bundle, R.style.AppTheme)

    // THEN - Task details are displayed on the screen
    // make sure that the title/description are both shown and correct
    onView(withId(R.id.task_detail_title_text)).check(matches(isDisplayed()))
    onView(withId(R.id.task_detail_title_text)).check(matches(withText("Active Task")))
    onView(withId(R.id.task_detail_description_text)).check(matches(isDisplayed()))
    onView(withId(R.id.task_detail_description_text)).check(matches(withText("AndroidX Rocks")))
    // and make sure the "active" checkbox is shown unchecked
    onView(withId(R.id.task_detail_complete_checkbox)).check(matches(isDisplayed()))
    onView(withId(R.id.task_detail_complete_checkbox)).check(matches(not(isChecked()))))
}
```

Homework 2: Write another integration test

Write an integration test that for the details of a *completed* task (named `completedTaskDetails_DisplayedInUi`)

- Check that the **title** & **description** are both shown and correct
- Check that the **checkbox** is shown checked

Do NOT change any code other than this test function (the variables like repository shouldn't be changed)

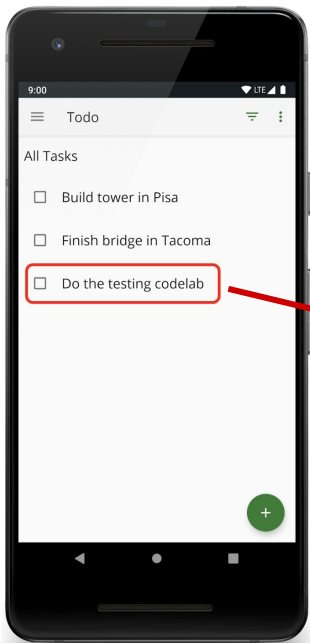
Today's Goal

- Check that when *“an active task's fragment is launched, its details are displayed on screen”*.
 - Integration of TaskDetailsFragment and TaskDetailsViewModel
- Check that *“pressing one of the tasks in the TasksFragment navigates to the clicked task's detail screen”*.
 - Integration of TasksFragment and Navigation component

Test plan

- GIVEN – the home screen with two tasks
- WHEN – we click on the first item
- THEN – verify that we called the navigate function to move to the first task's detail screen

Understanding Navigation in Android



1 User taps a task in the TasksFragment

2 TasksViewModel.openTask(id) is called...

```
TaskDetailFragmentTest.kt  TasksFragment.kt  task_item.xml  build.gradle.kts (app)
16 <layout xmlns:android="http://schemas.android.com/apk/res/android"
19 <data>
20 </data>
21 </layout>
22
23 <LinearLayout
24     android:layout_width="match_parent"
25     android:layout_height="?android:attr/listPreferredItemHeight"
26     android:padding="10dp"
27     android:orientation="vertical">
28     <include layout="@layout/task_item" />
29 </LinearLayout>
30
31
32
33
34
35
36
37
38
39
40
41
```

3 ..which eventually triggers TaskDetailFragment.openTaskDetails(id)...

```
private fun openTaskDetails(taskId: String) {
    val action = TasksFragmentDirections.actionTasksFragmentToTaskDetailFragment(taskId)
    findNavController().navigate(directions = action)
}
```

4 ...that calls findNavController().navigate(action) to navigate to the TaskDetailFragment screen.

The difficulty of testing navigation

Navigation is a complex side effect:

- It doesn't return a value.
- It changes the UI indirectly.
- The destination fragment (TaskDetailFragment) is created behind the scenes by the Navigation component.

So, we can't assert:

```
assertEquals(currentFragment, TaskDetailFragment)
```

The Solution — Mocking the NavController

Instead of checking the screen, we mock the navigation behavior and only check if the navigation *action* was called.

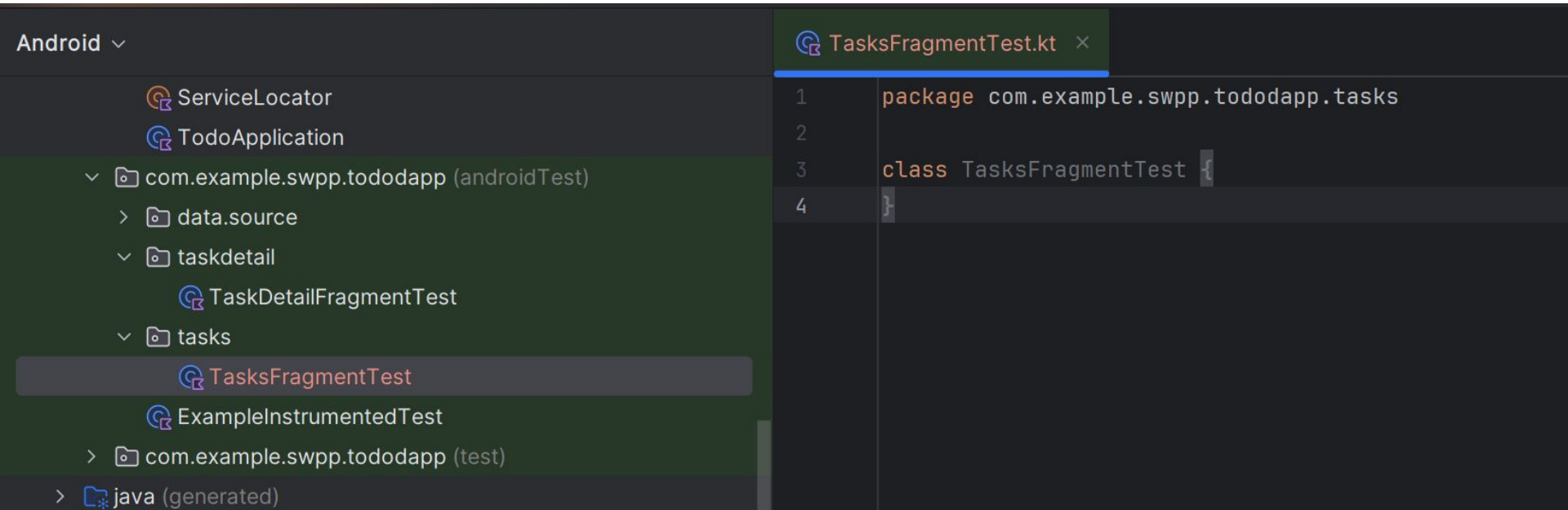
We replace the real NavController with a **mock object**.

```
verify(mockNavController)  
    .navigate(srcFragmentDirections.actionTasksFragmentToDstFragment(arg))
```

Then we verify that the `navigate()` method was called with the correct arguments.

Writing our navigation test

Create TasksFragmentTest the same way we did before



Writing our navigation test (1/2)

```
@RunWith(AndroidJUnit4::class)
@MediumTest
@ExperimentalCoroutinesApi
class TasksFragmentTest {
    private lateinit var repository: TasksRepository

    @Before
    fun initRepository() {
        repository = FakeAndroidTestRepository()
        ServiceLocator.tasksRepository = repository
    }

    @After
    fun cleanupDb() = runBlockingTest {
        ServiceLocator.resetRepository()
    }
}
```

Initialize with app *setup*
and *cleanup* code

Writing our navigation test (2/2)

Add two random tasks to our repository

```
@Test
fun clickTask_navigateToDetailFragmentOne() = runBlockingTest {
    repository.saveTask(Task("TITLE1", "DESCRIPTION1", false, "id1"))
    repository.saveTask(Task("TITLE2", "DESCRIPTION2", true, "id2"))
    // GIVEN - On the home screen with two tasks

    // WHEN - Click on the first list item

    // THEN - Verify that we navigate to the first detail screen
}
```

GIVEN – Home screen with two tasks

Create a mock NavController to track calls to this component.

```
val navController = mock(NavController::class.java)
```

```
val scenario = launchFragmentInContainer<TasksFragment>(Bundle(), R.style.AppTheme)
```

Creates a test environment (called FragmentScenario) and launches fragment in this test container

```
scenario.onFragment {  
    Navigation.setViewNavController(it.view!!, navController)}
```

Attach the fake controller to the fragment so that any navigate() call goes through our mock.

WHEN – Click on the first item (1/2)

Finds the RecyclerView of id tasks_list

```
onView(withId(R.id.tasks_list))
```

```
.perform(
```

```
    RecyclerViewActions.actionOnItem<RecyclerView.ViewHolder>(  
        hasDescendant(withText("TITLE1")), click())
```

Tell espresso to perform an action

WHEN – Click on the first item (2/2)

Perform an action on a specific item of the entire RecyclerView

```
onView(withId(R.id.tasks_list))
```

```
.perform(
```

```
RecyclerViewActions.actionOnItem<RecyclerView.ViewHolder>{
```

```
hasDescendant(withText("TITLE1")), click())
```

The specified item's type = ViewHolder

Find an item with TITLE1

The action = click

THEN – Check navigate to the detail screen

Mockito's verify to check the call **navigate** happened in navController

```
verify(navController).navigate(  
    TasksFragmentDirections.actionTasksFragmentToTaskDetailFragment( "id1")  
)
```

Arguments to the **navigate** call specifying the navigation destination (TaskDetailFragment with "id1" as argument)

Full code

```
@Test
fun clickTask_navigateToDetailFragmentOne() = runTest {
    repository.saveTask(Task("TITLE1", "DESCRIPTION1", false, "id1"))
    repository.saveTask(Task("TITLE2", "DESCRIPTION2", true, "id2"))
    // GIVEN - On the home screen with two tasks
    val scenario = launchFragmentInContainer<TasksFragment>(Bundle(), R.style.AppTheme)
    val navController = mock<NavController::class.java>
    scenario.onFragment { Navigation.setViewNavController(it.view!!, navController)}
    // WHEN - Click on the first list item
    onView(withId(R.id.tasks_list))
        .perform(
            RecyclerViewActions.actionOnItem<RecyclerView.ViewHolder>(
                hasDescendant(withText("TITLE1")), click()))
    // THEN - Verify that we navigate to the first detail screen
    verify(navController).navigate(
        TasksFragmentDirections.actionTasksFragmentToTaskDetailFragment( "id1")
    )
}
```

Homework 3: Write another navigation test

Write a test `clickAddTaskButton_navigateToAddEditFragment` that checks that if you click on the **+** button, you navigate to the `AddEditTaskFragment`.

Do NOT change any code other than this test function (the variables like `repository` shouldn't be changed)

Submission

- Submit 1 zip file on eTL named
 - 한국이름_학번.zip
 - Screenshot of test coverage (entire project)
Individual submission even for exercise 1
 - **TasksDetailFragmentTest.kt** with your
completedTaskDetails_DisplayedInUi function
 - **TasksFragmentTest.kt** with your
clickAddTaskButton_navigateToAddEditFragment function
- Deadline: **10/30 23:59**
- Late submission deadline: 10/31 23:59 (20% deduction)

Name	Date modified	Type	Size
Today			
TasksFragmentTest.kt	10/23/2025 9:49 AM	KT File	
TaskDetailFragmentTest.kt	10/23/2025 8:35 AM	KT File	
Earlier this week			
Screenshot	10/21/2025 5:03 PM	JPG File	

Recap

- Write integration test for “active task’s details are displayed on screen”.
 - Espresso
- Write integration test for “pressing one of the tasks in the TasksFragment navigates to the clicked task’s detail screen”.
 - Espresso with Mockito
 - Navigation

Cheat Sheet for Espresso

onView(ViewMatcher)
.perform(**ViewAction**)
.check(**ViewAssertion**);

onData(ObjectMatcher)
.DataOptions
.perform(**ViewAction**)
.check(**ViewAssertion**);



View Matchers

USER PROPERTIES

```
withId(...)
withText(...)
withTagKey(...)
withTagValue(...)
hasContentDescription(...)
withContentDescription(...)
withHint(...)
withSpinnerText(...)
hasLinks()
hasEllipsizedText()
hasMultiLineText()
```

UI PROPERTIES

```
isDisplayed()
isCompletelyDisplayed()
isEnabled()
hasFocus()
isClickable()
isChecked()
isNotChecked()
withEffectiveVisibility(...)
isSelected()
```

OBJECT MATCHER

```
allOf(Matchers)
anyOf(Matchers)
is(...)
not(...)
endsWith(String)
startsWith(String)
instanceOf(Class)
```

HIERARCHY

```
withParent(Matcher)
withChild(Matcher)
hasDescendant(Matcher)
isDescendantOf(Matcher)
hasSibling(Matcher)
isRoot()
```

INPUT

```
supportInputMethods(...)
hasInputAction(...)
```

CLASS

```
isAssignableFrom(...)
withClassName(...)
```

ROOT MATCHERS

```
isFocusable()
isTouchable()
isDialog()
withDecorView()
isPlatformPopup()
```

SEE ALSO

Preference matchers
Cursor matchers
Layout matchers

Data Options

```
inAdapterView(Matcher)
atPosition(Integer)
onChildView(Matcher)
```

View Actions

CLICK/PRESS

```
click()
doubleClick()
longClick()
pressBack()
pressBackButton()
pressKey(List<EspressoKey>)
pressMenuKey()
closeSoftKeyboard()
openLink()
```

GESTURES

```
scrollTo()
swipeLeft()
swipeRight()
swipeUp()
swipeDown()
```

TEXT

```
clearText()
typeText(String)
typeTextInfofocusedView(String)
replaceText(String)
```

View Assertions

```
matches(Matcher)
doesNotExist()
selectedDescendantMatch(...)
```

LAYOUT ASSERTIONS

```
noEllipsizedText(Matcher)
noMultiLineButtons()
noOverlaps(Matcher)
```

POSITION ASSERTIONS

```
isLeftOf(Matcher)
isLeftOf(Matcher)
isLeftAlignedWith(Matcher)
isRightAlignedWith(Matcher)
isAbove(Matcher)
isBelow(Matcher)
isBottomAlignedWith(Matcher)
isTopAlignedWith(Matcher)
```

intended(IntentMatcher);

intending(IntentMatcher)
.respondWith(ActivityResult);

Intent Matchers

INTENT

```
hasAction(...)
hasCategories(...)
hasData(...)
hasComponent(...)
hasExtra(...)
hasExtras(Matcher)
hasExtraKey(...)
hasType(...)
hasPackage()
toPackage(String)
hasFlag(int)
isInternal()
```

URI

```
hasHost(...)
hasParamWithName(...)
hasPath(...)
hasParamWithValue(...)
hasScheme(...)
hasSchemeSpecificPart(...)
```

BUNDLE

```
hasEntry(...)
hasKey(...)
hasValue(...)
```

COMPONENT NAME

```
hasClassName(...)
hasPackageName(...)
hasShortClassName(...)
hasMyPackageName()
```

v2.1.0, 4/21/2015

Useful Resources

- [Everything you need to know about Espresso](#) (look at other pages in the sidebar)
 - Basics
 - Testing AdapterView in Espresso
 - Debugging in Espresso
 - Idling resources: asynchronous operations that affect subsequent operations in a UI test
 - Espresso recipes: setup for common Espresso tests

Thank You.

Any Questions?